Initial Results: Two-Tier Control Plane for 6G Slicing

Dataset: network slicing dataset (Farreras et al. 2024, Zenodo 10610616). Prototype-first, 1-month scope.

1. Pipeline steps

1. Load a sample: per-slice offered load (eMBB / URLLC / mMTC) + global delay/loss.

2. Tier 1 (fast ML): linear forecaster predicts next aggregate load; numpy MLP predicts delay; p_violation from delay vs budget.

3. Analytic capacity model converts per-slice load -> required replicas.

4. Payload = per-slice KPIs + ML signals + current replicas -> Tier 2 LLM.

5. LLM proposes per-slice replica counts (MCP typed tool, or YAML manifest).

6. Verification gate checks schema + policy, then applies.

7. Hindsight oracle (offline, greedy P95) scores the decision vs future load.

2. Models

Tier 1 (fast ML): linear forecaster (window 5) + numpy MLP regressor (load -> delay), no sklearn. Baseline: predict-then-threshold.

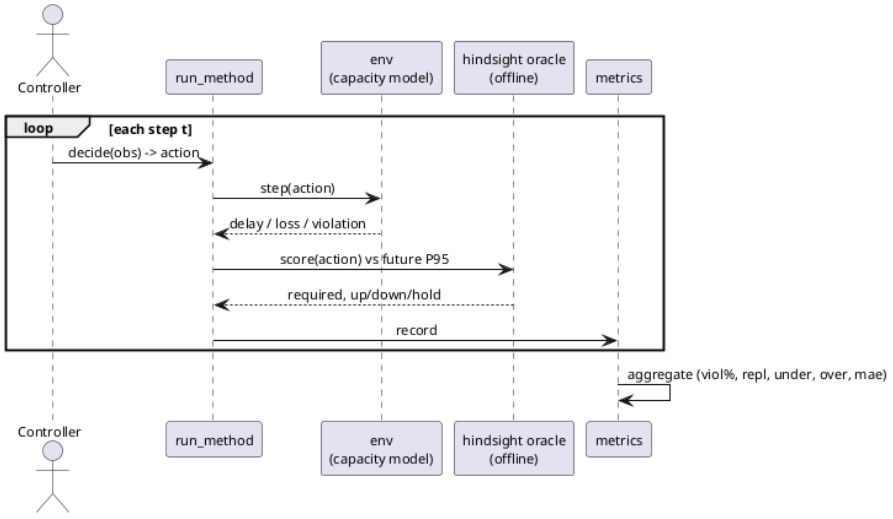
Tier 2 (LLM, Ollama cloud over localhost, OpenAI-compatible): glm-5.3-flash:cloud, glm-5.3:cloud, deepseek-v4-pro:cloud. Temperature 0, JSON output.

Actuation: MCP typed tools (estimate_capacity, scale_replicas, set_resources, add_nf_instance, apply_k8s_manifest).

Ground truth: analytic capacity model + greedy P95 hindsight oracle (prescient: looks at future load; offline only).

3. Method and controllers (detailed)

Harness: every controller is the same decide(obs) -> action. run_method feeds the action to the env and the oracle; metrics() aggregates.



Environment / capacity model

- state per slice = {load, replicas, delay, loss}
- action = {slice: replicas}, clamped to [1, 5]

- effective_capacity = replicas × cap_per_replica
- rho (utilization) = load / effective_capacity
- delay = base_delay / (1 − rho) when rho < 1, else max_delay
- loss = (load − capacity) / load when overloaded, else 0
- violation = delay > delay_budget OR loss > loss_budget
- NFConfig per slice: URLLC tight delay (4× base), eMBB loose (30× base), mMTC strict loss (0.5%)

Hindsight oracle (offline)

- at step t, look at future per-slice load over horizon H=5 (offset d=1)
- required = ceil(P95 / (cap × target_util))
- label up/down/hold vs the chosen action
- prescient — used only for scoring, never shown to the system

threshold

- reactive — acts only after a violation
- always one step late

```
if delay > budget: replicas = required(load)
elif delay < 0.5*budget and replicas > min: replicas -= 1
else: hold
```

hpa

- ideal controller — always matches the capacity model (mae 0)
- residual violation = the max_replicas=5 cap at peak load

```
replicas = required(load) = ceil(load / (cap * target_util))
```

predict-then-threshold

- linear forecaster predicts next aggregate load
- numpy MLP (load -> delay) predicts delay, then a threshold decides
- degenerates to HPA: the MLP over-predicts delay, so it almost always scales up

```
pred_delay = mlp(forecaster(history))
if pred_delay > budget: scale every slice to required(load)
elif pred_delay < 0.5*budget: scale down
else: hold
```

capacity-rule

- deterministic, no-network stand-in for the real LLM; same payload in, fixed rule out
- numbers are not hand-set: the switch-case picks the action, the env + oracle produce the metrics
- shares the required_replicas capacity model with the oracle, so it is a near-optimal upper-bound reference — intentionally close to the solution, not a competitor to beat

```
if p_violation > 0.5 and required > current: replicas = required
elif p_violation < 0.1 and current > min: replicas -= 1
else: hold
```

llm-real

- glm-5.3-flash:cloud via Ollama (OpenAI-compatible, localhost, no token), temperature 0
- receives the per-slice payload + ML signals, outputs JSON {actions, reason}
- ~6 s per decision
- conservative: over-provisions (over=0.96) to keep violation at 7.7%

4. Metrics used and why

Metric	Why it matters
SLA violation %	Outcome: how often delay/loss exceeds the slice budget (the cost the operator cares about).
mean replicas	Resource cost: over-provisioning wastes compute.
under-scaling	Missed scale-ups (replicas below oracle requirement) - the key small-model failure.
over-scaling	Unnecessary scale-ups (replicas above oracle requirement).
scale-up recall / precision	Decision quality: does the controller scale up when it should (recall) and only then (precision).
hold correctness	Does the controller leave well-enough alone when no change is needed.
MAE vs intended	Fidelity: distance from the ideal capacity-model decision (Mode T).
latency / tokens	Tier-2 cost: wall-clock and token cost of the LLM decision (RQ3).

5. Benchmark results (per-slice closed loop)

Scenario: **ramp**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	24.67	2.50	1.11	0.11	0.12	0.32	0.46	0.49
hpa	15.67	2.63	0.95	0.08	0.10	0.21	0.57	0.00
predict-then-threshold	15.67	2.63	0.95	0.08	0.10	0.21	0.57	0.00
capacity-rule	1.00	3.73	0.16	0.38	0.02	0.09	0.97	1.10

Scenario: **steady**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	27.50	2.58	1.28	0.22	0.12	0.30	0.17	0.59
hpa	16.67	2.63	1.13	0.12	0.09	0.19	0.40	0.00
predict-then-threshold	16.67	2.63	1.13	0.12	0.09	0.19	0.40	0.00
capacity-rule	0.83	4.28	0.12	0.76	0.11	0.22	0.96	1.65

Scenario: **spike**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	20.00	2.15	1.11	0.19	0.11	0.36	0.52	0.44
hpa	14.58	2.24	0.95	0.12	0.15	0.33	0.62	0.00
predict-then-threshold	15.00	2.23	0.96	0.12	0.14	0.31	0.62	0.01
capacity-rule	1.67	4.06	0.22	1.21	0.09	0.33	0.95	1.82

Scenario: **ramp-llm (34 states, includes real LLM)**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	28.43	2.49	1.47	0.10	0.20	0.55	0.28	0.50
hpa	20.59	2.52	1.41	0.07	0.18	0.43	0.32	0.00
predict-then-threshold	20.59	2.52	1.41	0.07	0.18	0.43	0.32	0.00
capacity-rule	2.94	3.55	0.49	0.18	0.11	0.36	0.88	1.03
llm-real-flash	8.82	3.63	0.81	0.58	0.14	0.21	0.58	1.26

6. RQ5 — does the estimate_capacity tool help?

Same model (glm-5.3-flash:cloud), same 20 states, with vs without tool-calling. Tool mode routes calls through the MCP server (estimate_capacity + scale_replicas).

mode	viol%	repl	mae	under	over

no_tool	3.33	3.53	1.22	0.50	0.48
tool	10.00	2.57	0.22	1.12	0.13

Finding: the tool cuts MAE 1.18 -> 0.20 and cost 3.47 -> 2.55 replicas (the LLM defers to the model), but raises violation 6.7% -> 11.7% because the model's 0.7 target utilization is tighter than the LLM's own headroom. The tool makes the LLM exact, not safer.

7. LLM calls (4 per model)

model	call	eMBB	URLLC	mMTC	lat(ms)	tokens	reason
glm-5.3-flash:cloud	1	1	2	3	2702.40	1178	mMTC violates delay (0.00426
glm-5.3-flash:cloud	2	3	4	1	3525.00	1482	eMBB and URLLC both violate
glm-5.3-flash:cloud	3	3	4	1	1907.20	968	eMBB delay ~1.7x target & lo
glm-5.3-flash:cloud	4	3	4	1	3108.10	1346	eMBB delay 1.67x target with
glm-5.3:cloud	1	1	2	5	4347.20	1299	mMTC severely violates KPIs
glm-5.3:cloud	2	3	4	1	6143.40	1694	eMBB and URLLC badly violate
glm-5.3:cloud	3	4	5	1	4336.80	1191	eMBB and URLLC massively vio
glm-5.3:cloud	4	4	5	1	3402.90	1140	eMBB and URLLC both breach d
deepseek-v4-pro:cloud	1	1	1	3	13055.70	2779	mMTC KPI loss/delay violatio
deepseek-v4-pro:cloud	2	5	5	1	6373.70	1602	eMBB and URLLC severely viol
deepseek-v4-pro:cloud	3	5	5	1	3884.90	1140	High p_violation and severe
deepseek-v4-pro:cloud	4	5	5	1	5389.60	1422	eMBB and URLLC are severely

8. Comparison graphs

